

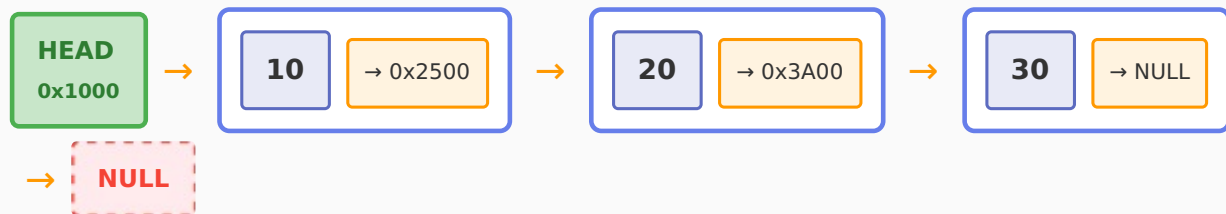
🔗 LinkedList - Complete Visual Guide

1 What is a LinkedList?

A **LinkedList** is a linear data structure where elements (called **nodes**) are stored in separate memory locations and connected using **pointers**. Unlike arrays, elements are not stored in contiguous memory locations.

💡 **Think of it like a treasure hunt where each clue (node) tells you where to find the next clue!**

Basic LinkedList Structure



📦 **Data** = The actual value stored

➡ **Next Pointer** = Memory address of the next node

⚡ **NULL** = End of the list

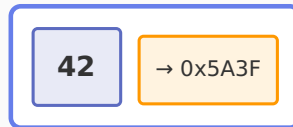
📍 **Address (e.g., 0x1000)** = Memory location where node is stored

2 Understanding a Node

Each **node** in a linkedlist contains two parts:

1. **Data**: The value you want to store
2. **Next**: A pointer/reference to the next node

Single Node Anatomy @ 0x2A4C



🔗 The "next" pointer stores the **memory address** (like 0x5A3F) of where the next node is located in RAM

```
// Node structure in Java

class Node {
    int data;
    Node next;

    // Constructor
    public Node(int data) {
        this.data = data;
        this.next = null;
    }
}

// Generic version (can store any type)
class Node<T> {
    T data;
    Node<T> next;

    public Node(T data) {
        this.data = data;
        this.next = null;
    }
}
```

3 Detailed Memory Representation

Let's see how LinkedList nodes are actually stored in memory with their addresses

RAM Memory View

Address	Node Contents	Notes
HEAD	Points to → 0x1000	Start of list
0x1000	Data: 10 Next: 0x2500	First node
0x1500	Other data / Unused memory	
0x2500	Data: 20 Next: 0x3A00	Second node
0x3000	Other data / Unused memory	
0x3A00	Data: 30 Next: NULL	Last node

🔍 Notice how the nodes are NOT stored consecutively!
Node 1 is at 0x1000, Node 2 is at 0x2500, and Node 3 is at 0x3A00.
Each node's "next" pointer contains the memory address where the next node lives.

4 Time Complexity Comparison

Understanding the performance characteristics of different operations

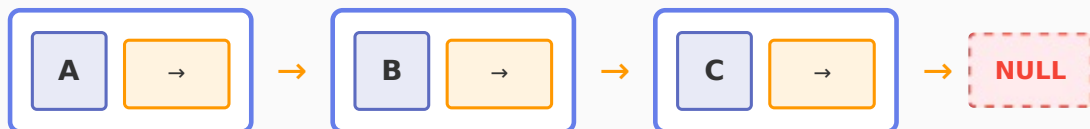
Operation	Array	LinkedList	Winner
Access by Index	$O(1)$	$O(n)$	🏆 Array
Insert at Beginning	$O(n)$	$O(1)$	🏆 LinkedList
Insert at End	$O(1)^*$	$O(n)$	🏆 Array (if space available)
Insert in Middle	$O(n)$	$O(1)^{**}$	🏆 LinkedList (if position known)
Delete	$O(n)$	$O(1)^{**}$	🏆 LinkedList (if position known)
Search	$O(n)$	$O(n)$	🤝 Tie
Memory Usage	Less	More (pointers)	🏆 Array

* Array insertion at end is $O(1)$ only if there's space; otherwise $O(n)$ due to resizing

** LinkedList insertion/deletion is $O(1)$ only if you already have the pointer to that position

5 Types of LinkedLists

Singly LinkedList

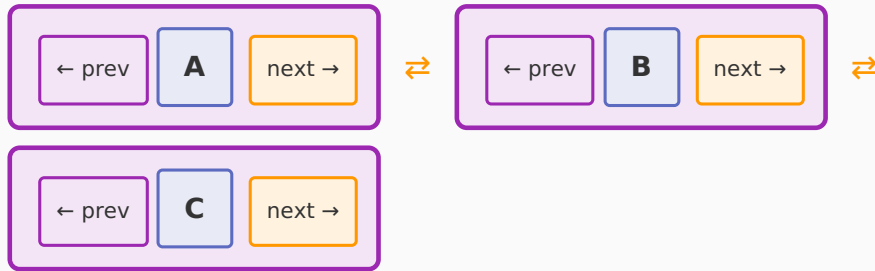


One-way travel: Can only move forward

Use case: Simple lists, stacks, queues

Doubly LinkedList

Each node has TWO pointers: **prev** (previous) and **next**



Two-way travel: Can move forward and backward

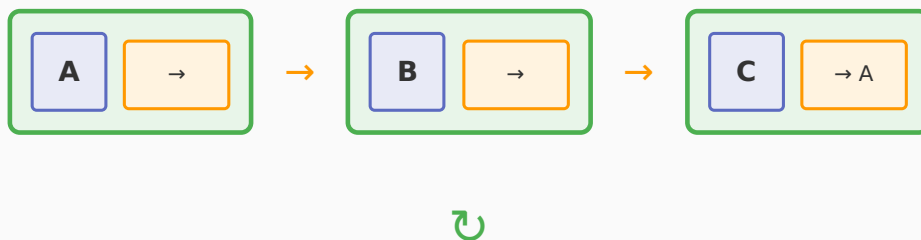
Use case: Browser history (Back/Forward), undo/redo functionality

```
// Doubly LinkedList Node in Java
class DoublyNode {
    int data;
    DoublyNode prev; // Points to previous node
    DoublyNode next; // Points to next node

    public DoublyNode(int data) {
        this.data = data;
        this.prev = null;
        this.next = null;
    }
}
```

Circular LinkedList

Last node points back to the first node (forms a circle)



Last node (C) points back to first node (A)

Never-ending loop: Last connects to first

Use case: Round-robin scheduling, multiplayer game turns, music playlists

```
// Creating a Circular LinkedList
lastNode.next = head; // Point last node back to head
```

6 Complete Java Implementation

Here's a simple LinkedList implementation in Java

```

// Node class
class Node {
    int data;
    Node next;

    public Node(int data) {
        this.data = data;
        this.next = null;
    }
}

// LinkedList class
class LinkedList {
    Node head;

    public LinkedList() {
        this.head = null;
    }

    // Insert at beginning - O(1)
    public void insertAtBeginning(int data) {
        Node newNode = new Node(data);
        newNode.next = head;
        head = newNode;
    }

    // Insert at end - O(n)
    public void insertAtEnd(int data) {
        Node newNode = new Node(data);

        if (head == null) {
            head = newNode;
            return;
        }

        Node current = head;
        while (current.next != null) {
            current = current.next;
        }
        current.next = newNode;
    }

    // Delete node - O(n)
    public void deleteNode(int key) {
        Node current = head;

        // If head needs to be deleted
        if (current != null && current.data == key) {
            head = current.next;
            return;
        }

        // Search for the node to delete
        Node prev = null;
        while (current != null && current.data != key) {
            prev = current;
            current = current.next;
        }

        if (current == null) {
            return;
        }

        prev.next = current.next;
    }

    // Print list - O(n)
    public void printList() {
        Node current = head;
        while (current != null) {
            System.out.print(current.data + " -> ");

```

```

        current = current.next;
    }
    System.out.println("NULL");
}

// Main method - Usage Example
public static void main(String[] args) {
    LinkedList ll = new LinkedList();
    ll.insertAtEnd(10);
    ll.insertAtEnd(20);
    ll.insertAtBeginning(5);
    ll.printList(); // Output: 5 -> 10 -> 20 -> NULL
}
}

```

7 Advantages & Disadvantages

✓ Advantages

- **Dynamic Size:** Grows and shrinks easily without pre-allocation
- **Easy Insertion/Deletion:** Just change pointers ($O(1)$ at known position)
- **No Memory Waste:** Allocate memory only as needed
- **Flexible:** Can implement stacks, queues, and graphs

✗ Disadvantages

- **No Random Access:** Must traverse from start ($O(n)$ to reach middle)
- **Extra Memory:** Additional storage needed for pointers
- **Cache Unfriendly:** Scattered memory locations hurt performance
- **Traversal:** Always $O(n)$ to reach the end

8 Real-World Use Cases

- 🌐 **Browser History:** Uses Doubly LinkedList (Back/Forward buttons)
- ↶ **Undo/Redo:** Track changes in text editors using Doubly LinkedList
- 🎵 **Music Playlist:** Circular LinkedList for continuous playback
- 🖼️ **Image Viewer:** Doubly LinkedList for next/previous navigation
- 📞 **Call Queue:** Singly LinkedList for First-In-First-Out (FIFO)
- 🎮 **Game Turn System:** Circular LinkedList for round-robin player turns

Quick Summary

- ✂ **LinkedList** = Chain of nodes connected by pointers
- 📦 **Node** = Data + Pointer to next node
- ✂ **Fast insertion/deletion** at known positions ($O(1)$)
- 🐢 **Slow random access** - must traverse ($O(n)$)
- 🔄 **Three types**: Singly, Doubly, Circular
- 💾 **Dynamic memory** - grows/shrinks as needed
- 📍 **Memory addresses** - nodes stored at scattered locations
- 🕒 **Best for**: Frequent insertions/deletions, unknown size
- ✖ **Avoid when**: Need fast random access